Davide Pozzebon

Modelli e Software per *L'Ottimizzazione Discreta*

Appunti del Corso

Ingegneria Informatica – UniPD

Anno Accademico 2026–2027

Davide Pozzebon

Modelli e Software per l'Ottimizzazione Discreta – Appunti

Descrizione dei contenuti

Questo lavoro è un resoconto dettagliato del corso di *Modelli e Software per l'Ottimizzazione Discreta* della laurea triennale in Ingegneria Informatica presso l'Università degli Studi di Padova.

Sorgente e altri appunti

Il documento originale è reperibile, insieme anche agli appunti di altri corsi del curriculum della laurea, dalla repository GitHub al link seguente: github.com/DavidePoz/Ingegneria-Informatica—Appunti

INDICE

1 Nozioni introduttive **4**
- 1.1 Modellare la realtà . . . 4
- 1.2 Modelli matematici e ottimizzazione . . . 5
- 1.3 Paradigmi di ottimizzazione . . . 6
  - 1.3.1 Programmazione Lineare . . . 6
  - 1.3.2 Programmazione Lineare Intera . . . 7
  - 1.3.3 Programmazione con Vincoli . . . 7
  - 1.3.4 Soddisfacibilità booleana . . . 7

2 Concetti generali dell'ottimizzazione **9**
- 2.1 Nozioni paradigma-indipendenti . . . 9
  - 2.1.1 Inferenza . . . 9
  - 2.1.2 Ricerca . . . 9
  - 2.1.3 Rilassamento . . . 10
  - 2.1.4 Dualità . . . 11
- 2.2 Approcci di ricerca . . . 12
  - 2.2.1 Ricerca naive: Generate-And-Test . . . 12
  - 2.2.2 Ricerca ad albero: Branch-and-Bound . . . 12

3 Programmazione Lineare **15**
- 3.1 Forma standard e forma canonica . . . 15
  - 3.1.1 Formulazioni equivalenti . . . 15
- 3.2 Interpretazione geometrica . . . 16
  - 3.2.1 Descrizione interna ed esterna . . . 17
  - 3.2.2 Teorema fondamentale della PL . . . 18
- 3.3 Esempi di modellizzazione con PL . . . 19
  - 3.3.1 Problema del trasporto . . . 20
  - 3.3.2 Problema della dieta . . . 21
- 3.4 Dualità . . . 22
  - 3.4.1 Formulazione matematica del duale . . . 22
  - 3.4.2 Approccio e lemma di Farkas . . . 23
  - 3.4.3 Dualità debole e dualità forte . . . 25
  - 3.4.4 Esempio . . . 27

4 Programmazione Lineare Intera **29**
- 4.1 Interpretazione geometrica . . . 29
- 4.2 Legame con la PL . . . 30
  - 4.2.1 Approssimazione della struttura ideale . . . 30

4.2.2 Ricerca ad albero . . . 32
4.2.3 Branch and Cut . . . 33
4.3 Modellazione PLI/MIP . . . 35
4.3.1 Decisioni binarie . . . 35
4.3.2 Variabili discrete . . . 35
4.3.3 Costi fissi . . . 36
4.3.4 Disgiunzioni . . . 37
4.3.5 Funzione linare a tratti . . . 38
4.4 Rappresentabilità MIP . . . 39
4.4.1 Problemi SAT . . . 41
4.4.2 Max-Cut . . . 41
4.4.3 Casi patologici . . . 42

5 Problemi di modellazione 44
5.1 Knapsack . . . 44
5.2 Facility location . . . 44
5.3 Set covering . . . 45

1 NOZIONI INTRODUTTIVE

Definizione (OTTIMIZZAZIONE MATEMATICA). L'**ottimizzazione** (o programmazione matematica, PM) è una branca della matematica applicata che studia **teoria e metodi** per la **ricerca dei punti di massimo e minimo di una funzione** all'interno di un dominio specificato.

Per cui, per poter applicare l'ottimizzazione matematica abbiamo bisogno di:

1. Un **insieme di decisioni** (che costituiranno la funzione da ottimizzare).
2. Poter **misurare il risultato delle decisioni**.
3. Poter **confrontare la "bontà" delle decisioni**.

1.1 Modellare la realtà

In generale, l'ottimizzazione matematica è utilizzata per **trovare soluzioni ottimali** a **problemi reali** (in ambito ingegneristico, industriale, economico-finanziario etc.).

Per prima cosa, abbiamo quindi bisogno di **creare un modello** che **descriva il problema reale**; poi procederemo a **risolverlo** e, infine, dovremmo **interpretare la soluzione** trovata:

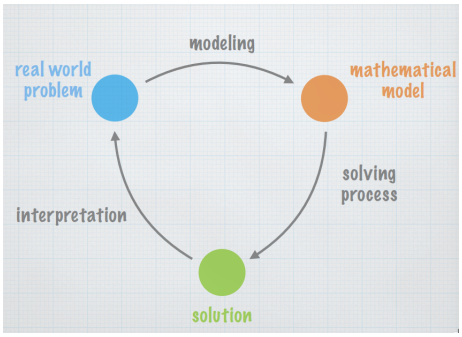


Figura 1.1: Schema di applicazione dell'ottimizzazione matematica

Attenzione. La fase di **interpretazione** è **spesso sottovalutata**: è **importantissimo** interpretare i risultati per capire *cosa rappresentano* e, soprattutto, se *sono compatibili con la realtà*.

1.2 Modelli matematici e ottimizzazione

In generale, i modelli matematici sono delle formulazioni del problema P del tipo:

Definizione (FORMULAZIONE PROBLEMA DI OTTIMIZZAZIONE).

$$\begin{aligned} &\min/\max f(x) \\ &S \\ &x \in D \end{aligned}$$

Dove $f(x)$ è la **funzione obiettivo** nelle variabili reali $x = (x_1, \ldots, x_n)$, $D = D_1 \times \ldots \times D_n$ è il dominio di x, ed S è un **insieme finito di vincoli**.

In genere si considerano problemi di ottimizzazione in cui vogliamo minimizzare la funzione obiettivo, ma possiamo "convertire" un modello in una o nell'altra forma a piacimento. Chiariamo ora alcuni dettagli:

Definizione (OTTIMIZZAZIONE DISCRETA). Se il dominio D è **discreto**, si parla di **ottimizzazione discreta**.

Definizione (OTTIMIZZAZIONE COMBINATORIA). Se il dominio D è **discreto e finito**, allora si parla di **ottimizzazione combinatoria**.

Definizione (SOLUZIONE). Ogni $x \in D$ è **soluzione** del problema P.

Definizione (VINCOLO). Un vincolo $c \in S$ è una **funzione** associata ad un **sottoinsieme di variabili** x_c, il cui valore può essere **vero** (in tal caso il vincolo è **soddisfatto**), oppure **falso** (in tal caso il vincolo è **violato**).

Definizione (SOLUZIONE AMMISSIBILE). Una **soluzione** che **rispetti tutti i vincoli** si dice **ammissibile**.

L'**insieme** delle soluzioni ammissibili si indica con $F(P)$.

Definizione (SOLUZIONE OTTIMA). Una soluzione ammissibile x^* si dice **ottima** se

$$f(x^*) \leq f(x)\ , \forall x \in F(P)$$

Inoltre:

Definizione (PROBLEMA IMPOSSIBILE). Un problema di ottimizzazione si dice **impossibile** (*infeasible*) se **non ammette alcuna soluzione ammissibile**, cioè $F(P) = \emptyset$

Definizione (PROBLEMA ILLIMITATO). Un problema di ottimizzazione si dice **illimitato** (*unbounded*) se **non esiste** alcun **limite inferiore** a $f(x)$ per $x \in F(P)$.

Definizione (PROBLEMA RISOLTO). Un problema di ottimizzazione si dice **risolto** quando ne viene **trovata una soluzione ottima** (dimostrando che è tale), oppure quando si **dimostra che il problema è impossibile o illimitato**.

E' inoltre importante chiarire il seguente dettaglio:

Attenzione (MODELLAZIONE VS RISOLUZIONE). In questo corso ci concentreremo sulla fase di **modellizzazione**. Questo perchè:

- Gli *algoritmi risolutivi* sono *incredibilmente complessi* e sono diventati 4-5 ordini di grandezza più efficienti nel corso degli ultimi 50 anni; studiarli da zero rappresenterebbe solo un handicap.
- La fase di modellizzazione è comunque molto importante come strumento a disposizione di un ingegnere.

1.3 Paradigmi di ottimizzazione

Purtroppo, un problema di ottimizzazione **arbitrario** è **indecidibile**: è quindi necessario aggiungere dei vincoli alla *"forma del problema"*.

Chiaramente, imporre dei vincoli ai modelli significa anche **ridurne l'espressività** (ovvero possiamo rappresentare fedelmente meno problemi reali); tuttavia esistono due paradigmi di ottimizzazione che rappresentano un **buon compromesso** tra *espressività* e *risolubilità*:

1. **Programmazione Lineare**(PL) - trattata nel corso.
2. **Ottimizzazione Convessa** - non trattata nel corso.

Nota (PROGRAMMAZIONE LINEARE). Nella programmazione lineare, la **funzione obiettivo** ed i **vincoli** sono **lineari** (duh!).

Per quanto stringente possa sembrare questa limitazione, moltissimi problemi possono essere modellati in questo modo e, inoltre, si tratta di una branca assai matura.

Di seguito vediamo più nel dettaglio alcuni paradigmi di nostro interesse.

1.3.1 Programmazione Lineare

Un problema di **programmazione lineare** (LP) consiste nella minimizzazione di una **funzione lineare** soggetta ad un numero finito di **vincoli lineari**. In generale si ha la forma:

Definizione (PROBLEMA DI PROGRAMMAZIONE LINEARE).

$$\begin{aligned} \min\ & c^\top x \\ & a_i^\top x \sim b_i\ , i = 1, \dots, m \\ & l_j \leq x_j \leq u_j\ , j = 1, \dots, n \end{aligned}$$

Con $\sim \in \{\leq, =, \geq\}$, $l_j \in \mathbb{R} \cup \{-\infty\}$ e $u_j \in \mathbb{R} \cup \{+\infty\}$.

Chiariamo ora alcuni dettagli sulle variabili nella definizione:

Nota (RUOLO DELLE VARIABILI NELLA DEFINIZIONE). Le variabili nella definizione hanno i seguenti ruoli:

- Le variabili $x_1, \dots, x_n$ compongono il **vettore delle decisioni** x.
- $c \in \mathbb{R}^n$ *pesa le variabili* di nostro interesse per ottenere la *funzione obiettivo* $c^\top x$.
- I vettori $a_i^\top \in \mathbb{R}^n$ *pesano ogni variabile* per formalizzare i *vincoli* $i = 1, \dots, m$.
- Il vettore $b = (b_1, \dots, b_m) \in \mathbb{R}^m$ completa la formalizzazione dei vincoli.

1.3.2 Programmazione Lineare Intera

Un problema di **programmazione lineare intera**(Mixed Integer Programming) consiste nella minimizzazione diuna funzione lineare soggetta ad un numero finito di vincoli lineari, con in più il vincolo che **alcune variabili** devono assumere **valori interi**.

Definizione (PROBLEMA MIP).

$$\begin{aligned} \min\ & c^\top x \\ & a_i^\top x \sim b_i\,, i = 1, \dots, m \\ & l_j \le x_j \le u_j\,, j = 1, \dots, n \\ & x_j \in \mathbb{Z}\,, \forall j \in J \subseteq N = \{1, \dots, n\} \end{aligned}$$

Con $\sim \in \{\le, =, \ge\}$, $l_j \in \mathbb{R} \cup \{-\infty\}$ e $u_j \in \mathbb{R} \cup \{+\infty\}$ come prima.

Da notare che se $J = N$ si parla di *programmazione lineare intera* **pura** (**tutte** le variabili sono **intere**), mentre se $J = \emptyset$ si ha a che fare con un problema di programmazione lineare classica.

1.3.3 Programmazione con Vincoli

Un problema di **programmazione con vincoli** è definito da una **terna** (X, D, C), dove X è un insieme di variabili con **dominio finito** D, e C è un insieme finito di **vincoli di natura arbitraria**.

Nota. Il paradigma CP è **molto più libero** nella definizione dei vincoli che si possono usare per modellare un problema. Questo fa si che spesso la modellazione sia più immediata e compatta.

Di contro, se i vincoli usati **non** hanno una **struttura** che **permetta lo sviluppo di algoritmi efficienti**, la **capacità di risoluzione** sarà al contrario **penalizzata**.

1.3.4 Soddisfacibilità booleana

Un problema di **soddisfacibilità booleana** è un caso particolare di problema di *soddisfacibilità*.

Definizione (PROBLEMA DI SODDISFACIBILITA' BOOLEANA). **Tutte le variabili** sono **booleane** (cioè possono assumere solo i valori *True* o *False* e i **vincoli** sono un numero finito di **clausole booleane**.

Come nel caso LP/MIP, in un problema SAT il **linguaggio a disposizione** per modellare un problema di ottimizzazione viene **notevolmente ristretto**.

> **Nota.** Nonostante ciò, molti problemi di notevole importanza pratica, come la **progettazione di circuiti logici**, possono essere espressi naturalmente con questo paradigma.

2 CONCETTI GENERALI DELL'OTTIMIZZAZIONE

2.1 Nozioni paradigma-indipendenti

La teoria e gli algoritmi sviluppati per affrontare problemi nei quattro paradigmi presentati sopra sono notevolemente diversi gli uni dagli altri; tuttavia esistono **alcuni concetti fondamentali** che sono **comuni a tutti gli approcci dell'ottimizzazione**, anche **indipendentemente dal paradigma**.

2.1.1 Inferenza

In un problema di ottimizzazione, può capitare che un **vincolo** C sia **dedotto** (derivato) da un **insieme di vincoli** S.

> **Definizione** (VINCOLO DEDOTTO). Se **ogni** $x \in D$ che **soddisfa** S **soddisfa anche** C, allora diciamo che C è un **vincolo dedotto**.

Chiaramente, dal punto di vista della modellazione, un tale vincolo è **ridondante** dato che **non cambia l'insieme delle soluzioni ammissibili**; tuttavia potrebbe rivelarsi (in alcuni casi) molto utile per **l'efficienza degli algoritmi di ricerca** in quanto potrebbe *"esplicitare" informazioni* che *non sono derivabili da nessun altro vincolo se preso singolarmente.*

> **Esempio.** Consideriamo ad esempio la seguente regione ammissibile:
>
> $$x_1 + x_2 \geq 1 \quad x_1 - x_2 \geq 0 \quad x_1, x_2 \in \{0, 1\}$$
>
> E' immediato verificare che $x_1 = 1$ è un **vincolo valido** per S, ma **non è immediatamente deducibile** dai due vincoli sopra se **presi separatamente**.

2.1.2 Ricerca

Un concetto molto importante è quello di ricerca:

> **Definizione** (RICERCA). Dato un problema P, la **ricerca** consiste nel **risolvere una serie di restrizioni**
>
> $$P_1, \ldots, P_m$$

di P.

Le **restrizioni** sono **ottenute aggiungendo dei vincoli** al problema originale.

In altre parole, rendiamo *più "stringente"* il problema andando ad aggiungere ulteriori vincoli. L'intuizione alla base delle restrizioni è che in alcuni casi potrebbe risultare *conveniente* aggiungere dei vincoli per **restringere lo spazio delle soluzioni**, in modo tale da rendere il problema **più facile da risolvere**.

In particolare, sono importanti le **ricerche esaustive**:

Definizione (RICERCA ESAUSTIVA). Una ricerca si dice **esaustiva** se l'insieme delle restrizioni considerate **"copre" tutto lo spazio** delle **soluzioni ammissibili** di P:

$$\bigcup_i F(P_i) = F(P)$$

Nel caso di una ricerca esaustiva, possiamo **rivolvere** P **risolvendo tutte le restrizioni** P_i e **scegliendo la soluzione migliore**.

In ogni caso, è importante il seguente fatto:

Proposizione. Data una restrizione P_i, una **sua qualsiasi soluzione ammissibile** è **ammissibile anche** per il problema di partenza P.

In particolare se $\tilde{x}$ è **soluzione ottima della restrizione**, essa rappresenta un **upper bound** per la **soluzione al problema originale** x^*:

$$x^* \leq \tilde{x}$$

L'intuizione dietro al fatto è la seguente:

Nota (INTUIZIONE). Le soluzioni di P_i sono un **sottoinsieme delle soluzioni originali**, per cui una soluzione di P_i deve **essere anche soluzione** di P.

Inoltre, se $\tilde{x}$ è **soluzione ottima della restrizione**, sappiamo che la **soluzione al problema originale** sarà **"almeno così tanto buona"**.

2.1.3 Rilassamento

Il *rilassamento* è il concetto complementare alla restrizione. Un rilassamento R è ottenuto da un problema di ottimizzazione P:

1. Eliminando alcuni vincoli.
2. Sostituendo la funzione obiettivo $f(x)$ con la sua *approssimazione inferiore* $g(x)$.

Più formalmente:

Definizione (RILASSAMENTO). R è un **rilassamento** di P se:

1. $F(P) \subseteq F(R)$
2. $g(x) \leq f(x)\ \forall x \in F(P)$

In maniera analoga a quanto detto per le restrizioni, l'idea è che a volte potrebbe risultare più semplice risolvere una *"versione rilassata"* del problema.

Attenzione. Sebbene si possano ricavare informazioni utili, bisogna fare attenzione al fatto che il **problema NON è equivalente**.

In tal senso, è utile considerare le seguenti proposizioni:

Proposizione. Dato un problema P ed un suo rilassamento R è possibile derivare che:

1. Se $F(R) = \emptyset$, allora P è **impossibile**.
2. Se x^* è **soluzione ottima** di R, $g(x^*)$ è un **lower bound** per il **valore ottimo** di P.
3. Se x^* è **soluzione ottima** di R, $x^* \in F(P)$ e $g(x^*) = f(x^*)$, allora x^* è **ottima** per P.

E' utile discutere i tre punti:

Nota. Possiamo intuire che:

1. Se il **rilassamento** (la versione "semplificata") è **impossibile**, lo **deve essere anche il problema originale**.
2. La soluzione ottima del rilassamento ci fornisce un **"tetto massimo"** per la **"bontà" della soluzione**: avendo "accesso" solo ad un sottoinsieme delle soluzioni, quella di nostro interesse sicuramente *non potrà essere migliore.*
3. Si tratta di un **caso fortunato**: se la soluzione del rilassamento è **ottima**, **ammisibile** per P, e i **valori delle funzioni obiettivo coincidono**, allora x^* è **ottima anche per** P.

2.1.4 Dualità

Un problema di ottimizzazione P viene spesso **interpretato come problema di ricerca** (problema *primario*): vogliamo **"cercare" la soluzione ottima**.

Alternativamente, lo stesso problema può anche essere visto come un **problema di inferenza** (problema *duale*): a partire dai vincoli, vogliamo **derivare il più stretto lower bound possibile** su $f(x)$.

Definizione (PROBLEMA DUALE). Il problema di **trovare la dimostrazione** che **porta al lower bound più stretto** prende il nome di **problema duale**.

Nota (INTERPRETAZIONE). In altre parole:

- Il problema **originale** cerca di **"scendere" verso il valore minimo** trovando **soluzioni ammissibili via via migliori**,
- Il problema **duale** cerca di **"salire"** costruendo un **limite inferiore sempre più alto**, manipolando i vincoli a disposizione.

Questo è un punto cardine della programmazione matematica, dato che permette di **certificare l'ottimalità di una soluzione**: quando il **valore ottenuto dalla ricerca** coincide con il **limite inferiore dimostrato dall'inferenza**, si ha la **garanzia** di aver trovato l'**ottimo globale**.

2.2 Approcci di ricerca

2.2.1 Ricerca naive: Generate-And-Test

Un possibile approccio per applicare la ricerca ad un problema di ottimizzazione è *generate-and-test*. Banalmente, l'idea di base è la seguente:

> **Definizione** (GENERATE-AND-TEST). Generiamo **tutte le soluzioni** possibili, poi le **confrontiamo** e **scegliamo la migliore**.

Formalmente, si tratta di considerare **tutte le restrizioni** in cui x è **fissato** (cioè tutti i suoi possibili valori); è quindi chiaro che questo approccio è applicabile solo in caso di problemi di **ottimizzazione combinatoria**, in cui il *numero di soluzioni è finito*.

E' importante notare che:

> **Attenzione.** La complessità computazionale di questo approccio è **sempre** $O(\beta^n)$, dove β è il numero di valori che possono assumere le variabili x_i.

Ciò non significa che non sia mai applicabile, vuol semplicemente dire che i suoi casi applicativi sono piuttosto *rari/particolari*. Se per esempio il problema coinvolgesse *poche variabili*, ma con *vincoli non lineari particolarmente complessi*, *"testare" tutte le soluzioni* potrebbe risultare *più conveniente che "risolvere i vincoli"*.

2.2.2 Ricerca ad albero: Branch-and-Bound

Un'approccio *"più strutturato"* è quello della **ricerca ad albero**. L'idea di base è la seguente:

> **Definizione** (RICERCA AD ALBERO). Vogliamo **partizionare** lo spazio delle **soluzioni possibili**, **iterando ricorsivamente** il metodo fino a quando otteniamo **partizioni sufficientemente facili da risolvere**.

L'idea è che *dopo una serie di iterazioni*, il problema sarà stato *scomposto abbastanza* da essere diventato *più facile da risolvere*, per una qualche definizione di "più facile da risolvere". E' tuttavia importante notare che:

> **Attenzione.** Al **caso peggiore**, questo algoritmo è **peggio di generate-and-test**. Si tratta del caso in cui *generiamo tutte le soluzioni* (come generate-and-test): in questo caso, avremmo anche l'*aggiunta di tutti i passaggi intermedi* (per ottenere le restrizioni).

Un'esempio di ricerca ad albero è data dall'algoritmo *Branch-and-Bound*, che usa una strategia **divide and conquer** per trovare la soluzione ottimale. In particolare, costruisce un **albero di restrizioni**, sulle quali **risolve dei rilassamenti** per determinare se è possibile eliminare dei rami.

> **Definizione** (BRANCH-AND-BOUND). Sia P un problema di ottimizzazione con funzione obiettivo $f(x) = c^\top x$ (da **minimizzare**), sia $\overline{x} \in F(P)$, e sia $z = f(\overline{x})$ un **upper bound** (iniziale) per il problema, detto **incumbent**.
>
> Sia A una **coda** di problemi inizializzata con P: $A = [P]$. **Finchè** A **non è vuota, si ripetono** le seguenti operazioni:
>
> 1. Si **sceglie un problema** P_i di A

2. Si **risolve un rilassamento** R_i di P_i, e:
   (a) Se **non ammette soluzione**, si può **eliminare il ramo**
   (b) Se la **soluzione del rilassamento** è **maggiore** dell'**incumbent**, si può **eliminare il ramo**.
   (c) Se la **soluzione** del rilassamento è **ammissibile** per il problema originale P ed è **minore** dell'**incumbement**, **aggiorniamo quest'ultimo** con il nuovo valore trovato.
3. Se il problema **non è stato scartato**, allora si **restringe ulteriormente**: $F(P_i) = \bigcup_j F(P_{ij})$.
4. Si **rimuove il problema** P_i (lasciando solo le sue restrizioni).

Vale la pena discutere le decisioni prese al punto due:

Nota. Le scelte prese al punto due derivano dai seguenti fatti:

1. Se il *rilassamento della restrizione considerata* è *impossibile* possiamo scartare quel *branch* perchè *sappiamo già* che anche *tutte le successive restrizioni sarebbero impossibili.*
2. Similmente, se la soluzione è *peggiore dell'incumbent* attuale, sappiamo già che *anche tutte le soluzioni delle successive restrizioni lo saranno.*
3. Se la soluzione trovata è *migliore di quella attualmente conosciuta*, allora essa diventa la nostra *nuova miglior stima* e possiamo *aggiornare l'incumbent.*

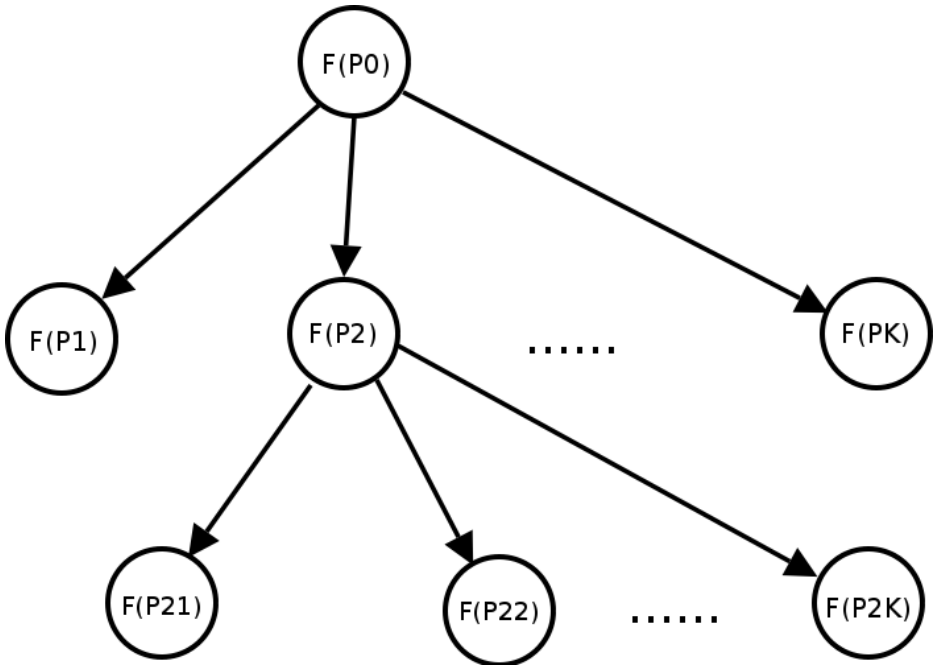


Figura 2.1: Esempio di albero associato all'esecuzione dell'algoritmo

E' inoltre importante tenere a mente che:

Nota (EFFICIENZA DELL'ALGORITMO). L'efficienza dell'algoritmo è **fortemente influenzata** da:

1. **Partizionamento del problema** nelle sue restrizioni.
2. **Scelta dei rilassamenti** associati alle restrizioni (bisogna generare *problemi semplici da risolvere*, ma che contrinuiscano con *lower bound ragionevoli*).

Infine, ricollegandoci a quanto detto sopra sugli algoritmi di ricerca ad albero al caso peggiore, sottolineaiamo che:

Nota. Branch and Bound è un *algoritmo di enumerazione implicita* perché si comporta esattamente come un *algoritmo di enumerazione* (cioè "prova" tutte le soluzioni possibili fino a trovare quella ottima), ma ne **scarta alcune** dimostrando a priori la loro *non ottimalità*.

3 PROGRAMMAZIONE LINEARE

Torniamo ora al paradigma della **programmazione lineare** per discuterlo più nel dettaglio.

3.1 Forma standard e forma canonica

Abbiamo già visto una possibile forma in cui può presentarsi un problema di programmazione lineare nel capitolo introduttivo.

Ora, senza perdita di generalità, possiamo assumere che il problema sia espresso in **forma standard**:

Definizione (FORMA STANDARD).

$$\begin{cases} \min c^\top x \\ Ax = b \\ x_i \geq 0 \end{cases}$$

Oppure in **forma canonica**:

Definizione (FORMA CANONICA).

$$\begin{cases} \min c^\top x \\ Ax \geq b \\ x_i \geq 0 \end{cases}$$

Nota. Il nome di queste forme deriva dal fatto che **qualsiasi altro problema di programmazione lineare** è **riconducibile a queste**.

3.1.1 Formulazioni equivalenti

Possiamo **passare** da una **formulazione generica** di un problema di programmazione lineare ad una **forma standard (o canonica) equivalente** applicando le regole di trasformazione seguenti:

1. Conversione da **massimizzazione a minimizzazione**

$$\max(c^\top x) = -\min(-c^\top x)$$

2. Conversione da **vincoli** "$\geq$" a vincoli "$=$"

$$a_i^\top x \geq b_i \iff \begin{cases} a_i^\top x - s_i = b_i \implies a_i^\top x = b_i' \\ s_i \geq 0 \end{cases}$$

3. Conversione da **vincoli** "$\leq$" a vincoli "$=$"

$$a_i^\top x \leq b_i \iff \begin{cases} a_i^\top x + s_i = b_i \implies a_i^\top x = b_i' \\ s_i \geq 0 \end{cases}$$

4. Conversione da **vincoli** "$=$" a vincoli "$\geq$"

$$a_i^\top x = b_i \iff \begin{cases} a_i^\top x \geq b_i \\ -a_i^\top x \geq -b_i \end{cases}$$

5. Conversione da **variabili libere** a **non negative**

$$x_i \text{ libera} \iff \begin{cases} x_i = x_i^+ - x_i^- \\ x_i^+, x_i^- \geq 0 \end{cases}$$

6. Conversione da **lower bound generico** a **variabile non negativa**

$$x_i \geq l_i \iff \begin{cases} x_i = x_i' + l_i \\ x_i' \geq 0 \end{cases}$$

7. Conversione da **upper bound generico** a **variabile non negativa**

$$x_i \leq u_i \iff \begin{cases} x_i = u_i - x_i' \\ x_i' \geq 0 \end{cases}$$

8. Conversione da **variabile bounded** a **variabili non negative**

$$l_i \leq x_i \leq u_i \iff \begin{cases} x_i = x_i' + l_i \\ x_i' + s_i = u_i - l_i \\ x_i', s_i \geq 0 \end{cases}$$

> **Attenzione.** Il **passaggio** da una forma all'altra può richiedere una **variazione del numero di variabili e/o vincoli** del problema; tuttavia le trasformazioni inducono un **blow-up al più polinomiale** della taglia del problema.

3.2 Interpretazione geometrica

Proviamo ora a dare un'interpretazione geometrica ad un problema di PL. Per cominciare, diamo le seguenti due definizioni:

Definizione (SEMISPAZIO AFFINE e IPERPIANO). Gli insiemi

$$\{x \in \mathbb{R}^n : a^\top x \leq a_0\}$$
$$\{x \in \mathbb{R}^n : a^\top x = a_0\}$$

Si dicono (rispettivamente) **semispazio affine** e **iperpiano** *indotti* da (a, a_0).

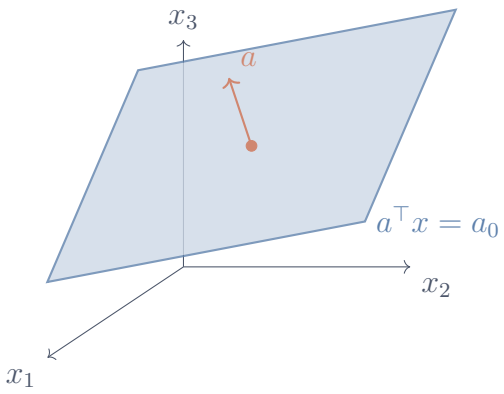


Figura 3.1: Esempio di iperpiano in $\mathbb{R}^3$

Definizione (POLIEDRO). Si dice **poliedro** l'**intersezione** di un **numero finito** di **semispazi affini** ed **iperpiani**.

Notiamo che equazioni di questo tipo sono *proprio quelle che compaiono in un problema di PL*. L'**insieme delle soluzioni ammissibili** di un problema di programmazione lineare è quindi, per definizione, un **poliedro**; più precisamente, un **politopo**:

Definizione (POLITOPO). Si dice **politopo** un **poliedro limitato**

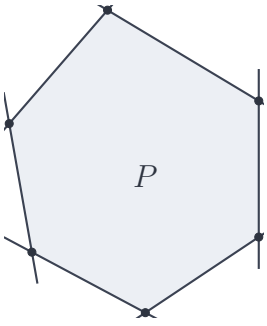


Figura 3.2: Esempio di politopo in $\mathbb{R}^2$

3.2.1 Descrizione interna ed esterna

Quella che abbiamo visto sopra è la **descrizione esterna** di un politopo, ovvero una sua descrizione tramite **intersezione di vincoli**.

Dalla figura 3.2 possiamo tuttavia intuire che è possibile **descrivere un politopo** tramite i suoi **vertici**: si parla in questo caso di **descrizione interna**.

Definizione (VERTICE). Si dice **vertice di un poliedro** P un **punto** x che **non** può essere **espresso** come **combinazione convessa stretta** di **altri due punti del poliedro** stesso. Ovvero, dato il punto $x^* \in P$, esso è **vertice** se:

$$\begin{aligned} &\forall x_1, x_2 \in P \\ &\nexists \lambda_1, \lambda_2 \in (0,1) : \lambda_1 + \lambda_2 = 1 \\ &: \lambda_1 x_2 + \lambda_2 x_2 = x \end{aligned}$$

Ogni politopo ha un *numero finito di vertici* e soddisfa il seguente teorema:

Teorema. Ogni **punto di un politopo** può essere espresso come **combinazione convessa** dei suoi **vertici** $\{x_1, \dots, x_t\}$:

$$x \in P \iff \begin{cases} x = \sum_{j=1}^{t} \lambda_j x_j \\ \sum_{j=1}^{t} \lambda_j = 1 \\ \lambda_j \geq 0 \,, \forall j = 1, \dots, t \end{cases}$$

Chiaramente, i vertici sono **tutti necessari** per descrivere un poliedro.

Ora, per facilitare la compresione della definizione del teorema, riportiamo alcune note sulle combinazioni convesse:

Nota (COMBINAZIONE CONVESSA). Una combinazione convessa di t punti è una loro **combinazione lineare** in cui:

- I **pesi** λ_j sono **non negativi**.
- I **pesi sommano a uno**.

Con **due punti**, le loro **combinazioni convesse** formano il **segmento di retta** che li unisce. In generale, la combinazione convessa di un insieme di punti (non allineati) **genera la figura convessa** (piena) che li **"avvolge"**, chiamata **involucro convesso** (*convex hull*).

Esempio (SEGMENTO DI RETTA). Poniamo di avere:

$$\begin{aligned} &\alpha_1 x_1 + \alpha_2 x_2 = 0 \\ &\alpha_1 + \alpha_2 = 1 \implies \alpha_2 = 1 - \alpha_1 \end{aligned}$$

Quindi:

$$\begin{aligned} \alpha_1 x_1 + (1 - \alpha_1) x_2 &= \alpha_1 x_1 + x_2 - \alpha_1 x_2 \\ &= x_2 + \alpha_1 (x_1 - x_2) \end{aligned}$$

3.2.2 Teorema fondamentale della PL

L'**utilità** della **descrizione interna** di un politopo sta nel seguente teorema:

Teorema (TEOREMA FONDAMENTALE DELLA PL). Se un problema di programmazione lineare $\min\{cx : x \in P\}$ **ammette ottimo finito**, allora **esiste un vertice** di P **ottimo**.

Facciamo attenzione a un dettaglio:

Attenzione. I vertici **non sono gli unci ottimi**, il teorema afferma semplicemente che, se l'ottimo della funzione obiettivo è finito, allora **sicuramente un vertice di P sarà ottimo**.

Per capire meglio, proviamo a considerare gli **insiemi di livello della funzione obiettivo**:

$$f(x) = l \in \mathbb{R}$$
$$\implies c^\top x = l$$

Gli insiemi di livello per la funzione obiettivo di un problema di PL sono degli **iperpiani**. Al variare di l otteniamo **infiniti iperpiani**, **tutti paralleli** tra loro (tutti perpendicolari al vettore normale c, che rimane invariato). Notiamo inoltre che c è proprio il **gradiente** di $f(x)$, per cui:

Nota (RISOLVERE UN PROBLEMA PL). Intuiamo che **minimizzare** $f(x)$ significa **spostarsi** negli insiemi di livello **in direzione** $-c$ (*opposta* al gradiente) fino a trovare la **migliore soluzione** all'**interno di P**.

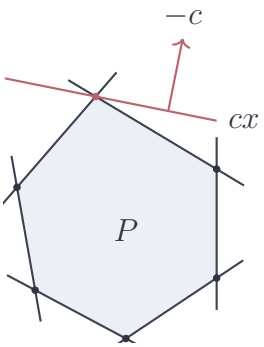


Figura 3.3: Interpretazione geometrica del teorema fondamentale della PL

La figura 3.3 illustra l'interpretazione discussa sopra. Ci permette inoltre di intuire perchè *non necessariamente tutti i punti ottimi sono vertici*; se infatti c dovessere essere ortogonale ad uno dei lati del politopo, allora *esisterebbe un insieme di livello coincidente con il lato in questione*, dando luogo a *infiniti punti ottimi* (di cui due sarebbero *comunque vertici*).

Inoltre, quello discusso è proprio l'approccio seguito da uno dei principali algoritmi di risoluzione per problemi PL:

Definizione (ALGORITMO DEL SIMPLESSO). Algoritmo risolutivo per problemi di PL che, **partendo da un vertice qualsiasi**, si **muove in modo greedy** ad un **vertice vicino non peggiore** ad ogni iterazione, **fino a trovare un vertice ottimo**.

3.3 Esempi di modellizzazione con PL

Per quanto problemi diversi saranno modellati diversamente, possiamo approcciare la fase di modellazione seguendo la seguente **scaletta generale**:

Nota (SCALETTA GENERALE). La scaletta è la seguente:

1. **Riconoscere** gli **insiemi** in gioco.

2. **Riconoscere** i **dati**(*parametri*, detti anche **attributi**) del problema (*dati in input*).
3. **Scegliere** le **variabili** (*decisioni* del problema).
4. Determinare la **funzione obiettivo**.
5. Determinare i **vincoli** del problema.

Di seguito vediamo alcuni esempi in cui il problema è modellato seguendo questa scaletta.

3.3.1 Problema del trasporto

Il problema è così posto:

Definizione (PROBLEMA DEL TRASPORTO). Abbiamo un **insieme di sorgenti** $\{s_1, s_2, \dots, s_n\}$ ed un **insieme di destinazioni** $\{d_1, d_2, \dots, d_k\}$.

- Per ogni coppia **sorgente-destinazione** (s_i, m_j) abbiamo un **certo costo** di spedizione c_{ij}.
- Ogni sorgente s_i ha una *certa quantità di bene* (**disponibilità**) d_i.
- Ogni destinazione m_j ha una *certa domanda* di prodotto (*richiesta*) r_j.

Vogliamo ideare il **piano di spedizione** che **minimizza il costo totale di spedizione**.

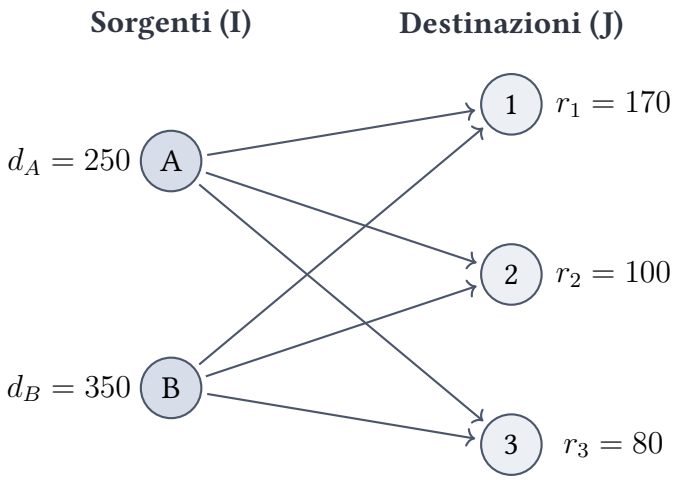


Figura 3.4: Esempio di problema del trasporto

Nota. La rappresentazione del problema è data da un **grafo bipartito**.

Nell'esempio in figura abbiamo a che fare con i seguenti insiemi:

$$I : \{\text{sorgenti}\} = \{A, B\}$$
$$J : \{\text{destinazioni}\} = \{1, 2, 3\}$$

Abbiamo inoltre i seguenti *dati di input* (**parametri**):

- **Disponibilità**: $d_A = 250$, $d_B = 350$
- **Richieste**: $r_1 = 170$, $r_2 = 100$, $r_3 = 80$.

- **Costi unitari** di spedizione: c_{ij}.

E scegliamo di utilizzare come **variabili**:

$$x_{ij} \in \mathbb{R}^+ = \text{ q.tà di prodotto da spedire da } i \text{ a } j \text{ , } (\forall i \in I \text{ , } \forall j \in J)$$

Dove con $\mathbb{R}^+$ intendiamo $x \in \mathbb{R}, x \geq 0$ (quindi 0 incluso).

> **Nota.** La *descrizione semantica* delle variabili scelte **non** *fa parte del modello*: si tratta semplicemente di una *descrizione utile alla compresione del modello*, ma, ovviamente, l'*algoritmo risolutivo non ne è a conoscenza.*

Dato che vogliamo **minimizzare il costo totale di spedizione**, ha senso scegliere la seguente **funzione obiettivo**:

$$f(x) = \sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij}$$

Infine, per quanto riguarda i vincoli, possiamo considerare i seguenti:

1. Le **richieste** delle destinazioni devono essere **soddisfatte**: $\sum_{i \in I} x_{ij} \geq r_j \ \forall j \in J$
2. **Non** bisogna **eccedere le disponibilità**: $\sum_{j \in J} x_{ij} \leq d_i \ \forall i \in I$

Terminato questo processo di modellazione, il nostro modello per il problema in esempio risulta quindi essere:

$$\begin{cases} \min \ \sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij} & \\ \sum_{i \in I} x_{ij} \geq r_j & \forall j \in J \\ \sum_{j \in J} x_{ij} \leq d_i & \forall i \in I \\ x_{ij} \in \mathbb{R}^+ & \forall i \in I \text{ , } \forall j \in J \end{cases}$$

3.3.2 Problema della dieta

Il problema è così posto:

> **Definizione** (PORBLEMA DELLA DIETA). Abbiamo:
>
> - Un **insieme di alimenti** $J = \{\text{alimenti}\}$ di una *possibile dieta* (per animali da fattoria da sfamare), con **costo unitario** c_j.
> - Una **lista di sostanze nutritive** $I = \{\text{sostanze nutritive}\}$, ciasuna con **quantità minima** b_i.
> - Dei valori a_{ij} che rappresentano la **quantità** di **nutriente** i che si trova **nell'alimento** j.

Scegliamo come **variabili**:

$$x_j \in \mathbb{R}^+ = \text{q.tà di elemento j nella dieta } \forall j \in J$$

Intuitivamente, il modello sarà:

$$\begin{cases} \min \sum_{j \in J} c_j x_j & \\ \sum_{j \in J} a_{ij} x_j \geq b_i & \forall i \in I \\ x_j \geq 0 & \forall j \in J \end{cases}$$

Nota. Vogliamo **minimizzare** il **costo della dieta** di un animale, rispettando comunque i **fabbisogni minimi** b_i per **ogni nutriente**.

3.4 Dualità

Ricordiamo che **risolvere un problema** di ottimizzazione **non** significa **solamente trovare una soluzione** ottima, è anche necessario **dimostrare** che la soluzione **sia effettivamente ottima**.

Tenendo questo a mente, riprendiamo il concetto di **dualità**. Abbiamo visto che un problema di ottimizzazione può essere visto come:

1. Un problema (**primale**) di **ricerca**: cerchiamo *soluzioni via via migliori* generando **nuovi upper bound**.
2. Un problema (**duale**) di **inferenza**: usiamo i *vincoli* per generare **lower bound** via via migliori.

Eseguendo **in parallelo** la risoluzione del **primario** e del **duale** possiamo **trovare una soluzione** e anche **dimostrarne l'ottimalità**.

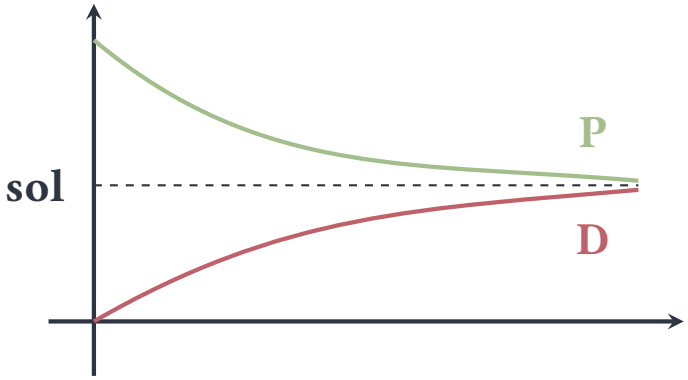


Figura 3.5: Risoluzione parallela del problema primario e del duale

3.4.1 Formulazione matematica del duale

Vediamo ora più nel dettaglio in cosa consiste il problema **duale**.

Dato un problema di ottimizzazione P del tipo

$$P : \begin{cases} \min c^\top x \\ Ax \geq b \end{cases}$$

possiamo intuire dalla figura e da quanto detto sopra che il suo **duale** consiste nel **determinare un lower bound** c_0 per $c^\top x$ e può essere formulato come segue:

Nota (FORMULAZIONE DEL DUALE). Risolvere il problema **duale** significa **cercare disuguaglianze** del tipo:

$$c^\top x \geq c_0$$

In modo che c_0 sia **massimo** (lower bound più stringente possibile). Cioè:

$$\begin{cases} \max c_0 \\ c^\top x \geq c_0 \end{cases}$$

A questo punto dobbiamo però precisare che la disuguaglianza che stiamo cercando deve **essere valida**. Formalizziamo allora questo concetto:

Definizione (DISUGUAGLIANZA VALIDA). Data una **regione ammissibile** P ed una **disuguaglianza** $\alpha^\top x \geq \beta$, diciamo che questa è **valida** in P **se e solo se** vale:

$$\alpha^\top x \geq \beta \; \forall x \in P$$

Verificare che la **disuguaglianza sia effettivamente rispettata** $\forall x \in P$ è chiaramente **infattibile** dato che esistono **infiniti punti** in P; per fortuna, però, possiamo sfruttare il seguente teorema:

Teorema. Se una **disuguaglianza** è **valida per tutti i vertici** di un politopo P, allora è valida **anche per tutti** gli **altri punti** di P.

Del resto, se P è identificato dai vertici $x_1, x_2, \ldots, x_t$, sappiamo che un punto $x \in P$ può essere scritto come:

$$x = \lambda_1 x_1 + \ldots \lambda_t x_t$$

Con $\sum \lambda_j = 1$ e $\lambda_j \geq 0$, $j = 1, \ldots, t$. Abbiamo quindi:

$$\begin{aligned} \alpha^\top x &= \alpha^\top (\lambda_1 x_1 + \ldots \lambda_t x_t) \\ &= \lambda_1 (\alpha^\top x_1) + \ldots + \lambda_t (\alpha^\top x_t) \end{aligned}$$

Supponiamo ora che la diseguaglianza sia **verificata per ogni vertice**:

$$\begin{aligned} \alpha^\top x_1 &\geq \beta \\ \alpha^\top x_2 &\geq \beta \\ &\ldots \\ \alpha^\top x_t &\geq \beta \end{aligned}$$

Allora risulta:

$$\begin{aligned} \alpha^\top x &\geq \lambda_1(\beta) + \ldots + \lambda_t(\beta) \\ &\Downarrow \\ \alpha^\top x &\geq \beta(\lambda_1 + \ldots + \lambda_t) = \beta \end{aligned}$$

Esattamente come volevamo □.

3.4.2 Approccio e lemma di Farkas

Abbiamo quindi capito che risolvere il problema duale significa cercare il **massimo** c_0 tale che $c^\top x \geq c_0$ sia **valida** per il problema P che stiamo risolvendo. Abbiamo inoltre capito *come* verificare se una disuguaglianza è *valida o meno*, ma non abbiamo ancora discusso **come trovare** c_0.

Nella pratica, la ricerca di c_0 si traduce nello **sfruttare i vincoli** $Ax \geq b$ per *dedurre* (**inferenza**) dei **lower bound stringenti** $c^\top x \geq c_0$.

Intuitivamente, possiamo pensare di ottenere tali lower bound **moltiplicando la disequazione** data dai **vincoli** per un **vettore** $u \geq 0$ (componenti *non negative*):

$$\underbrace{u^\top A}_{c} x \geq \underbrace{u^\top b}_{c_0}$$

Il problema diventa quindi:

$$\begin{cases} \max c_0 \\ c^\top x \geq c_0 \\ u \geq 0 \end{cases}$$

Nota (CONDIZIONI SU u)**.** Siccome il nostro problema **originale** aveva come **vincoli** $Ax \geq b$, dobbiamo **imporre** $u \geq 0$ per **non modificare il verso** della disuguaglianza.

Se **invece** i vincoli fossero stati della forma $Ax = b$, allora u sarebbe stato **libero**.

Ora, chiaramente **non possiamo procedere alla cieca**: u dovrà essere **scelto con criterio** affinchè la disuguaglianza **risulti valida**. Per questo, ci viene in aiuto il seguente lemma:

Teorema (LEMMA DI FARKAS)**.** Una **disuguaglianza** $c^\top x \geq c_0$ è **valida** per P **se e solo se**:

$$\exists u \in \mathbb{R}^m_+ \text{ t.c. } c^\top \geq u^\top A \text{ e } c_0 \leq u^\top b$$

Il lemma di Farkas ci permette di riscrivere il problema come segue:

$$\begin{cases} \max c_0 \\ c^\top \geq u^\top A \\ c_0 \leq u^\top b \\ u \geq 0 \end{cases}$$

Nota (CAMBIO DI PROSPETTIVA)**.** Notiamo che:

- Noi **vogliamo massimizzare** c_0.
- Contemporaneamente, il **vincolo** $c_0 \leq u^\top b$ impone un *upper bound* a c_0.

Chiaramente, la **scelta ottimale** è **far coincidere** c_0 con il suo *upper bound*, per cui imponiamo

$$c_0 \coloneqq u^\top b$$

E scegliamo di **massimizzare questa quantità**.

Alla luce di ciò, possiamo riscrivere il problema nella seguente forma:

$$\begin{cases} \max u^\top b \\ u^\top A \leq c^\top \\ u \geq 0 \end{cases}$$

Nota. Osserviamo che:

1. Il **duale** di un problema di PL è un **altro problema di PL**.
2. I **ruoli** dei **vettori** c e b si sono **scambiati**. Ciò significa che:
   (a) Il numero di **vincoli del duale** è uguale al numero di **variabili del primale**.
   (b) Il numero di **variabili del duale** è uguale al numeri di **vincoli del primale**.

Il secondo punto è schematizzato in figura.

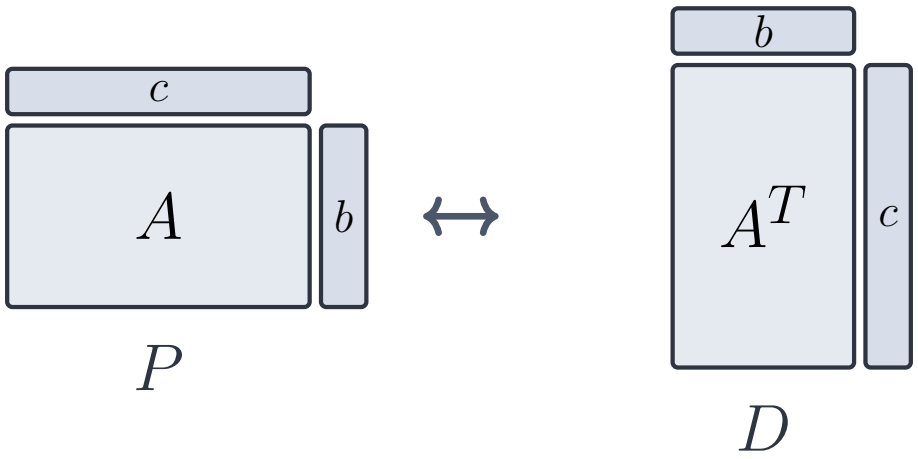


Figura 3.6: I ruoli delle variabili e dei vincoli nel duale sono scambiati

3.4.3 Dualità debole e dualità forte

Abbiamo visto che il problema **duale** può essere usato per produrre dei **lower bound** per il problema **primale**. Ciò è formalizzato dal concetto di **dualità debole**:

Proposizione (DUALITA' DEBOLE).

$$\forall x \in P\,, \forall u \in D$$
$$u^\top b \leq c^\top x$$

Cioè, per **qualsiasi sol.** x del problema **primale** P e per **qualsiasi sol.** u del problema **duale** D, il valore della **funzione obiettivo del duale** ($u^\top b$) sarà **sempre minore o uguale** al valore della **funzione obiettivo del primale**.

La dimostrazione parte dalle condizioni date dal lemme di Farkas ed è, molto banalmente, la seguente:

$$c^\top \geq u^\top A \implies c^\top x \geq u^\top A x \geq u^\top b$$

Inoltre, è importantissimo anche il concetto di **dualità forte**:

Proposizione (DUALITA' FORTE). Il **valore ottimo del duale** (d^*) **coincide** con il **valore ottimo del primale** (p^*) *SE E SOLO SE* **entrambi ammettono ottimi finiti**.

Possiamo interpretare questi due concetti come segue:

Nota (DUALITA' DEBOLE VS FORTE). La dualità forte è un concetto appunto *più forte* perchè *non solo* formalizza la *ricerca dei lower bound* per la soluzione, ma ci dice anche **quando soluzione del primale** e **soluzione del duale coincidono**.

Inoltre, questi concetti ci permettono di affermare che **solo alcune casistiche** riguardanti i **due problemi** sono effettivamente **ammissibili**. In figura sono schematizzate le situazioni ammissibili (contrassegnate da una spunta), e quelle *impossibili* (contrassegnate da una croce):

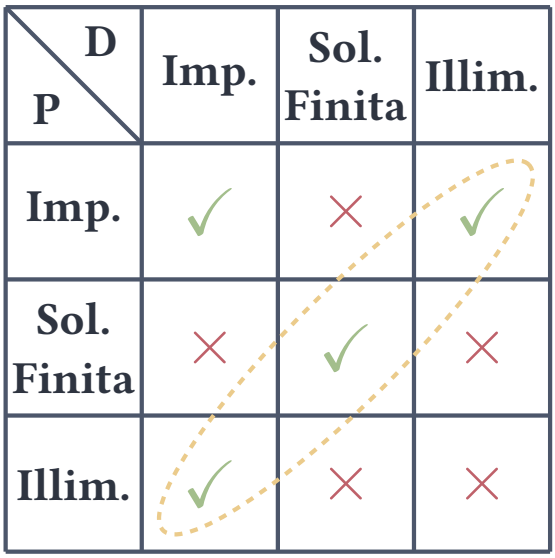


Figura 3.7: Casistiche ammesse dai concetti di dualità.

In figura sono evidenziati alcuni **casi interessanti** (contorno tratteggiato), cioè quelli che più spesso si manifestano negli esercizi.

In ogni caso, elenchiamo i casi possibili e la motivazione che sta dietro:

Nota. I seguenti casi sono possibili:

- **OTTIMO P - OTTIMO D** - Caso descritto dalla **dualità forte**.
- **INFINITO P - IMPOSSIBILE D** - Conseguenza della **dualità debole**.
- **IMPOSSIBILE P - INFINITO D** - Conseguenza della **dualità debole**.
- **IMPOSSIBILE P - IMPOSSIBILE D** - Conseguenza della **dualità debole**.

E' facile intuire che queste situazioni sono dirette conseguenze del fatto che la **soluzione del primale** rappresenta un **upper bound** per il problema, mentre quella del **duale** rappresenta un **lower bound**.

Inoltre, quando utilizziamo un software per risolvere un problema di PL, è bene tenere a mente:

Nota (SOTTO-PRODOTTO DELL'ALGORITMO). Quando usiamo un **algoritmo** (e.g. *algoritmo del simplesso*) per risolvere un problema, questo ci fornirà una **coppia di soluzioni**

$$(x^*, u^*)$$

Che sono, rispettivamente, **soluzione del primale** e **soluzione del duale**.

Ciò è una **conseguenza** del fatto che, come detto in precedenza, **non basta trovare una soluzione** per dire di aver **risolto il problema**, ma è anche necessario fornire la **dimostrazione di ottimalità**. Tale dimostrazione è data proprio dalla soluzione del duale.

3.4.4 Esempio

A questo punto, consideriamo alcuni semplici esempi per capire concretamente come operare con il problema duale.

Esempio (ESEMPIO DUALITA'). Consideriamo il seguente esempio:

$$\begin{cases} \min 4x_1 - 2x_2 + x_3 + 5x_4 & \text{f. obiettivo} \\ x_1 + x_2 \leq 1 & \text{vincolo} \\ 4x_2 - 3x_3 \geq 2 & \text{vincolo} \\ 2x_1 + x_3 - x_4 = 3 & \text{vincolo} \\ x_1, x_2 \geq 0 & \text{dom. variabili} \\ x_3 \leq 0 & \text{dom. variabili} \\ x_4 \text{ libera} & \text{dom. variabili} \end{cases}$$

Osserviamo subito che il problema ha **4 variabili** e **3 vincoli**, per cui ci aspettiamo che il **duale** abbia **3 variabili** e **4 vincoli**.

In particolare, abbiamo:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} \qquad c^\top = [4, -2, 1, 5]$$

$$b = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \quad A = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 4 & -3 & 0 \\ 2 & 0 & 1 & -1 \end{bmatrix}$$

Per cominciare, determiniamo le condizioni su $u^\top = (u_1, u_2, u_3)$ a partire dai vincoli:

$$\begin{aligned} x_1 + x_2 \leq 1 &\implies u_1 \leq 0 \\ 4x_2 - 3x_3 \geq 2 &\implies u_2 \geq 0 \\ 2x_1 + x_3 - x_4 = 3 &\implies u_3 \text{ libero} \end{aligned}$$

Inoltre, dai **termini noti dei vincoli** ricaviamo la **funzione obiettivo del duale**:

$$u^\top b = u_1 + 2u_2 + 3u_3$$

Che, ovviamente, vogliamo **massimizzare**.

Ora, per ottenere i vincoli dobbiamo considerare $u^\top A$ e $c^\top$. Per comodità, consideriamo invece $A^\top u$ e c:

$$\begin{bmatrix} 1 & 0 & 2 \\ 1 & 4 & 0 \\ 0 & -3 & 1 \\ 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix}, \quad \begin{bmatrix} 4 \\ -2 \\ 1 \\ 5 \end{bmatrix}$$

Abbiamo così *determinato anche i vincoli*, ed il **problema duale** risulta quindi essere:

$$\begin{cases} \max u_1 + 2u_2 + 3u_3 \\ u_1 + 2u_3 \leq 4 \\ u_1 + 4u_2 \leq -2 \\ -3u_2 + u_3 \geq 1 \\ -u_3 = 5 \\ u_1 \leq 0 \\ u_2 \geq 0 \\ u_3 \text{ libero} \end{cases}$$

Nota (VERSI DELLE DISEQUAZIONI NEI VINCOLI DUALI)**.** I **versi delle disequazioni** nei **vincoli** del **problema duale** sono ottenuti a partire dalle **condizioni sui domini** delle **variabili del primale**. In particolare:

- Un dominio del tipo $x_i \geq 0$ produce il **vincolo duale** $\leq$.
- Un dominio del tipo $x_i \leq 0$ produce il **vincolo duale** $\geq$.
- Un dominio del tipo x_i libero produce il **vincolo duale** $=$.
- Un dominio del tipo $x_i = 0$ produce il **vincolo duale** libero.

4 PROGRAMMAZIONE LINEARE INTERA

La formulazione generale di un problema di programmazione lineare intera è **molto simile** ad un **problema di PL**, con l'**aggiunta del vincolo di interezza** per *alcune* (in tal caso parliamo di *programmazione lineare mista*: MIP - Mixed Integer Programming) oppure *tutte* (in tal caso parliamo di *programmazione lineare intera pura*) le variabili del problema.

Definizione (FORMA GENERALE PROBLEMA PL INTERA). La **forma generale** di un **problema di programmazione lineare intera** è la seguente:

$$\begin{cases} \min/\max\ c^\top x & \\ a_i^\top x \sim b_i & i = 1, \dots, m\ ,\ \sim \in \{\leq, =, \geq\} \\ l_j \leq x_j \leq u_j & j = 1, \dots, n \\ x_j \in \mathbb{Z} & \forall j \in J \subseteq \{1, \dots, n\} \end{cases}$$

Anche se la **modifica** rispetto ad un problema di PL **sembra minima**, **in realtà** è **molto più complessa di quello che sembra**.

Nota (COMPLESSITA' PROBLEMI MIP). Questa tipologia di problemi è **NP-hard**.

4.1 Interpretazione geometrica

Per capire come mai la complessità di questa classe di problemi è maggiore, ricorriamo all'interpretazione geometrica.

Ricordiamo che l'**interpretazione geometrica** della **regione ammissibile** per un problema di PL era riconducibile ad un **poliedro**. Ora, invece, se assumiamo che il **vincolo di interezza** riguardi **tutte le variabili**, potremmo considerare solo i **punti all'interno del poliedro** che hanno **solo coordinate intere**, come mostrato nella figura 4.1:

Questa modifica alla regione ammissibile comporta alcune conseguenze:

Nota. Notiamo subito che:

1. La regioni ammissibile **non è più un insieme convesso**, il che *complica molto* la *risoluzione algoritmica del problema.*

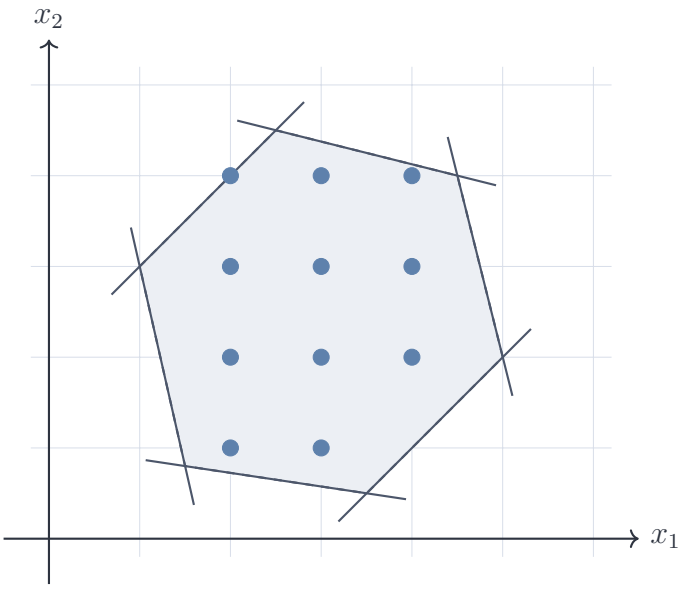


Figura 4.1: Interpretazione geometrica della programmazione lineare intera

2. **Non** è **più vero** che **esiste sempre un vertice ottimo**: addirittura, non è *nemmeno detto* che i *vertici appartengano alla regione ammissibile.*

4.2 Legame con la PL

Abbiamo assodato di **saper risolvere i problemi di PL**; ci viene allora spontaneo chiederci: **possiamo sfruttare questa conoscenza per risolvere i problemi MIP?**.

Una prima idea (*naive*) potrebbe essere considerare l'**inviluppo convesso ideale** (*ideal convex hull*) della regione di partenza, ovvero una *"sotto-regione"* il cui *bordo* passi per i *punti ammissibili "più esterni"*.

Per capire meglio di cosa si tratta, osserviamo la figura 4.2:

Nota (DIMENSIONE DEL PROBLEMA E LIMITAZIONI)**.** Per quanto questa idea possa sembrare **semplice e intuitiva**, risulta estremamente **difficile da applicare** a **problemi** in cui il **numero di decisioni è maggiore** di quello mostrato nell'esempio.

Di conseguenza, per molti problemi, questa **struttura** è **impossibile** (o comunque *non conveniente*) da **determinare**.

4.2.1 Approssimazione della struttura ideale

Notiamo che la **formulazione di un problema MIP**, in generale, **non è univoca**: possiamo avere **infinite formulazioni** che presentano tutte lo **stesso inviluppo**, come mostrato nella figura 4.3.

Questo fatto ci suggerisce una nuova idea: **cercare di approssimare** la **formulazione ideale** (*ideal convex hull*) in maniera **iterativa** fino al punto in cui troviamo un **problema di PL** la cui **soluzione risulta intera** (e quindi rispetta i vincoli del problema MIP).

Un algoritmo che implementa questa idea è **cutting planes** (*piani di taglio*), che funziona come descritto di seguito:

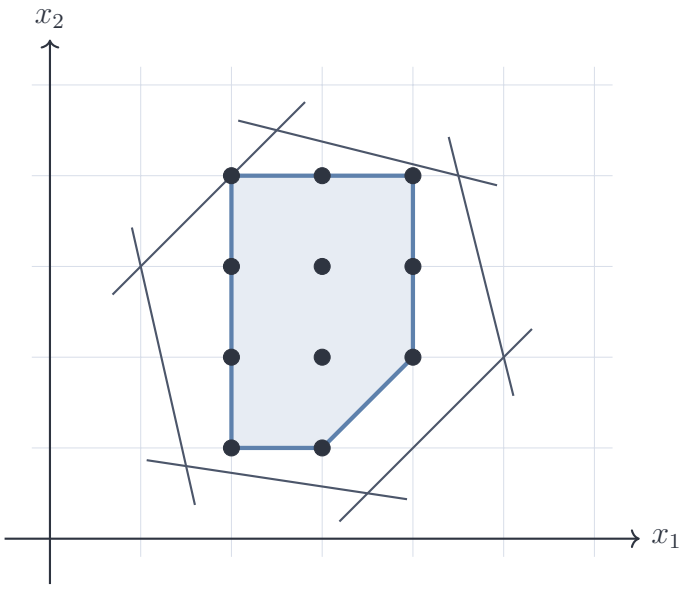


Figura 4.2: Ideal Convex Hull

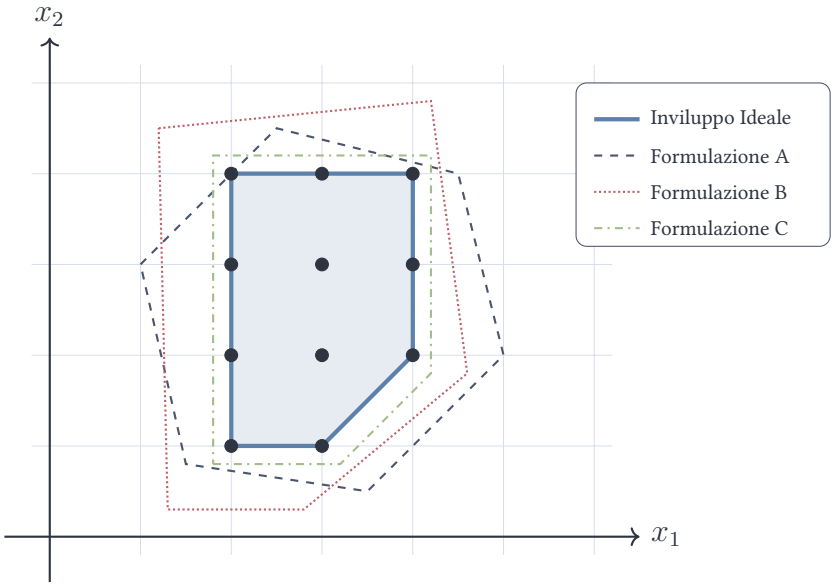


Figura 4.3: Diverse formulazioni possono portare allo stesso inviluppo

Definizione (ALGORITMO CUTTING PLANES)**.** Possiamo procedere come segue:

1. Considerare un **rilassamento continuo** del problema MIP (*senza vincoli di interezza*) e **determinarne la soluzione**.
   (a) Se la **soluzione trovata** ha **coordinate intere**, allora questa è anche *ottimo globale* e abbiamo **terminato**.
   (b) Altrimenti (se la soluzione *non rispetta i vincoli di interezza originale*) dobbiamo proseguire.

2. Operare una **separazione** (tramite **piano di taglio**), ovvero **introdurre un vincolo** che:
   (a) Ci permetta di **scartare la soluzione precedente** (*non ammissibile*).
   (b) **Risulti valido per tutte le soluzioni ammissibili** del nostro problema MIP.
3. **Ripetere questi step** a *partire dal problema rilassato* e *con il nuovo vincolo aggiunto.*

Sottolineaiamo il fatto che *noi non dobbiamo preoccuparci di come realizzare il punto 2*, ci *basta sapere che esistono algoritmi efficienti che lo implementano* e noi *ci limiteremo ad utilizzarli come "scatole nere"*.

Detto ciò, è tuttavia importante fare alcune osservazioni sull'**efficienza della soluzione**. Innanzitutto, come *tutti gli algoritmi risolutivi per problemi MIP*, la **complessità al caso pessimo** di **cutting planes** è **esponenziale**. Come se non bastasse, tuttavia, cutting planes **non** è **neanche particolarmente efficiente al caso medio** e, infatti, *non rappresenta lo stato dell'arte* nella *risoluzione dei problemi MIP*.

Nota (EFFICIENZA DI CUTTING PLANES). Intuitivamente, ciò è dovuto al fatto che, **dopo una certa soglia**, i **tagli sono sempre meno "efficienti"**, nel senso che sono *sempre più simili ai tagli precedenti.*

Osserviamo inoltre anche il seguente fatto (più interessante per noi):

Nota (OUTPUT DELL'ALGORITMO INTERROTTO). Se **interrompiamo l'esecuzione** dell'algoritmo, questo **non** avrà **prodotto alcuna soluzione ammissibile**, ma solo un **lower bound** (soluzione del rilassamento).

Questo *potrebbe essere utile o meno*, nel senso che **preferiremmo** ottenere una **soluzione ammissibile** (anche *non ottima*, ma *quantomeno ammissibile*).

4.2.2 Ricerca ad albero

Ricordiamo che avevamo menzionato gli algoritmi di **ricerca ad albero** (*branch and bound*) come esempio di metodo risolutivo dei problemi di ottimizzazione.

Possiamo applicare un'idea di questo tipo ai problemi MIP:

Definizione (RICERCA AD ALBERO). Possiamo procedere come segue:

1. Applichiamo un **algoritmo per PL** per determinare una soluzione x^*:
   (a) Se è **intera**, abbiamo **terminato**.
   (b) Altrimenti, **proseguiamo**.
2. Operiamo un **branching**. Per farlo, **aggiungiamo i vincoli** seguenti (per esempio sulla i-esima variabile) per ottenere **due restrizioni**:
$$x_i \leq \lfloor x_i^* \rfloor \quad x_i \geq \lceil x_i^* \rceil$$
3. **Ripetiamo** i passaggi in **maniera ricorsiva**.

L'intuizione geometrica dell'operazione di *branching* è mostrata nella figura 4.4, mentre l'albero delle restrizioni risultante è mostrato nella figura 4.5.

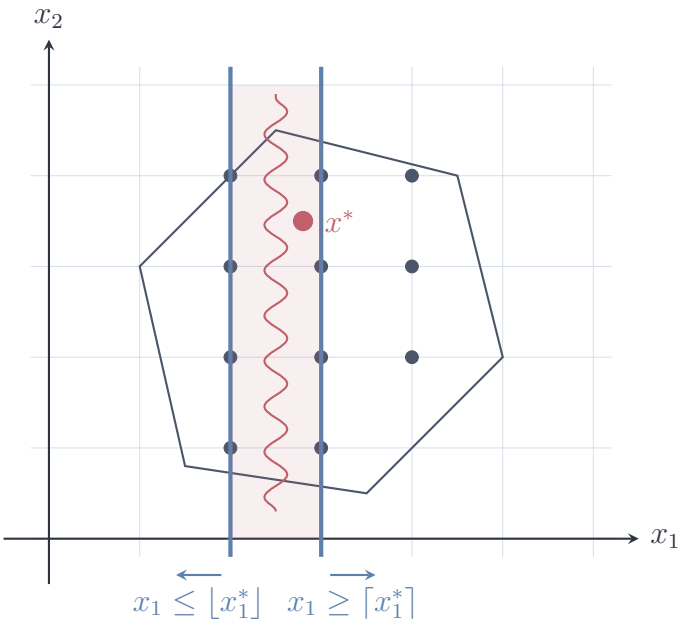


Figura 4.4: Ricerca ad albero applicata a problemi MIP

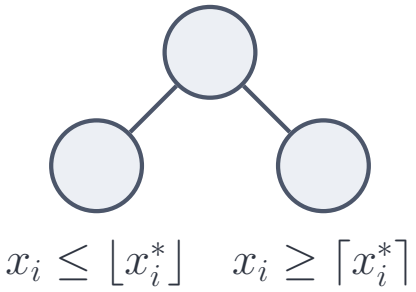


Figura 4.5: Nodi (restrizioni) risultanti dall'operazione di branching

> **Nota.** Al contrario dell'algoritmo *cutting planes*, se *durante la visita dell'albero* **troviamo una soluzione intera** e decidiamo di **interrompere l'esecuzione** dell'algoritmo, **otteniamo quantomeno una soluzione ammissibile**, anzichè solo un lower bound come nel caso di *cutting planes*.

4.2.3 Branch and Cut

Vediamo ora una **soluzione ancora più efficiente**, che **combina le due precedenti**. Le osservazioni chiave alla base sono le seguenti:

> **Nota** (OSSERVAZIONI ALLA BASE)**.** Notiamo che:
>
> 1. I **piani di taglio** ci permettono di **"migliorare" i rilassamenti**.
> 2. L'operazione di **branch** ci permette di *"cambiare strada"* quando i **tagli iniziano a "perdere efficienza"**.

Per cui l'algoritmo **branch and cut** risulta essere il seguente:

Definizione (BRANCH AND CUT). Le fasi da ripetere sono:

1. Risoluzione di un **rilassamento continuo**. Se la soluzione è *intera*, allora *abbiamo terminato*, altrimenti proseguiamo.
2. **Cut**: generiamo dei **piani di taglio** per ottenere **rilassamenti migliori**.
3. **Branch**: quando il processo di **cut inizia a perdere efficienza** eseguiamo un'operazione di **branch** per **produrre delle restrizioni**.
4. **Ripetiamo** la fase 1 **sulle restrizioni** ottenute.

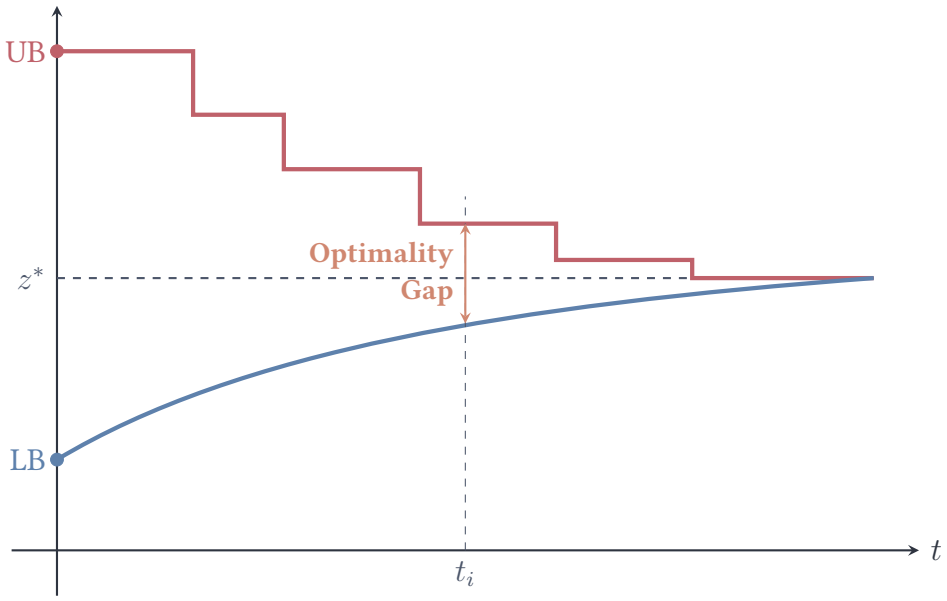


Figura 4.6: Progresso durante l'esecuzione di *Branch and Cut*

Proviamo ora a capire meglio *come opera questo algoritmo*, osservando la figura 4.6:

1. Dopo il **primo rilassamento**, abbiamo ottenuto un **primo lower bound**, che è **anche globalmente valido** in quanto risultato di un *rilassamento dell'"intero" problema originale.*
2. L'esecuzione di **algoritmi euristici** (*sui nodi, quindi sulle restrizioni*) genera **soluzioni ammissibili**, che fungono da **upper bound via via migliori** (*sequenza monotòna*).
3. I **lower bound** generati dalle **soluzioni dei rilassamenti** sono **validi solo** per la **restrizione corrispondente nell'albero** (lower bound *locali*). Per ottenere un **lower bound globale** dobbiamo **considerare il minor lower bound** tra quelli di ciascun **nodo aperto** (ciè il *peggior caso possibile* tra *tutte le strade aperte*).

Dunque, intuiamo che **interrompendo** prematuramente l'esecuzione dell'algoritmo, otteniamo come **prodotto**:

1. Una **soluzione ammissibile** z_{t_i} (la *migliore conosciuta fino a quel punto*).
2. Un **lower bound globale** l_{t_i} (*"certificato di qualità"*).
3. I due parametri sopra, insieme, formano ciò che chiamiamo **optimality gap**: $f(z_{t_i}) - l_{t_i}$.

Nota (CONSEGUENZE). Questo è molto importante (specialmente in rapporto alle soluzioni precedenti) in quanto:

- Il **prodotto dell'algoritmo** è molto **più informativo** rispetto ad avere *solo il lower bound* oppure *solo una soluzione.*
- **Maggiore** è il **tempo di esecuzione**, **migliore** è l'**optimality gap ottenuto**, e quindi anche la nostra **stima della soluzione ottima**.

4.3 Modellazione PLI/MIP

Vediamo ora come possono essere modellati i problemi della classe **PLI**. Per farlo, esistono **diverse formulazioni** da scegliere *a seconda del problema specifico.*

4.3.1 Decisioni binarie

Uno dei casi **più comuni** è quello in cui un problema viene ridotto ad una **lista di decisioni elementari** del tipo **yes/no**.

Definizione (DECISIONI YES/NO). Tali **decisioni** sono rappresentate tramite un **vettore di variabili binarie**:

$$x \in \{0,1\}^n$$

Dove n è il *numero di decisioni* associate al problema.

Intuitivamente, $x_i = 1$ significa che la **corrispondente decisione è affermativa**; viceversa, se $x_i = 0$, la *corrispondente decisione è negativa.*

4.3.2 Variabili discrete

Quando il problema *non può essere ridotto* a delle *semplici decisioni binarie* dobbiamo **gestire più valori**. Se questi sono **consecutivi**, allora la modellazione è molto *semplice.* Per esempio:

$$x \in \{2,3,4,5\} \implies 2 \leq x \leq 5\,,\, x \in \mathbb{Z}$$

Definizione (VARIABILI DISCRETE REGOLARI). In generale, basta considerare un **intervallo** ed il **vincolo di interezza**:

$$z_l \leq x \leq z_u\,,\, x, z_l, z_u \in \mathbb{Z}$$

Tuttavia è da notare che *molto spesso* i **valori possibili *non* presentano regolarità**. Consideriamo il seguente esempio:

$$y \in \{0.25, 0.5, 1.25, 3\}$$

Che possiamo immaginare associato (per esempio) ai *possibili valori di potenza erogata da un motore.*

La *formulazione di un problema* di ottimizzazione *richiede una serie di vincoli* in forma di *disequazioni*, che però *non possono rappresentare* un *insieme di questo tipo.* Dobbiamo allora ricorrere a delle **variabili di appoggio** che ci **permettano di effetuare la selezione** dei **valori ammissibili**.

Definizione (VARIABILI DISCRETE RIRREGOLARI). Dati i **valori ammissibili** per una variabile y_i:

$$y \in \{s_1, \dots, s_k\}$$

Definiamo le **variabili binarie di appoggio**:

$$x_i = \begin{cases} 1 & y = s_i \\ 0 & \text{altrimenti} \end{cases}$$

Per ottenere la formulazione seguente:

$$\begin{cases} y = \sum_{i=1}^{k} s_i x_i \\ \sum_{i=1}^{k} x_i = 1 \\ x_i \in \{0,1\} \qquad \forall i = 1, \dots, k \end{cases}$$

Nota (SELEZIONE DEL VALORE). Per come abbiamo **riformulato i vincoli**, notiamo che **uno ed uno solo** dei valori x_i sarà **pari ad uno**, per cui la **variabile** y assumerà il **corrispondente valore** s_i associato.

4.3.3 Costi fissi

Questo pattern si usa quando una *certa attività* comporta un **costo fisso di attivazione** (e.g. accendere un macchinario) **indipendente dalla quantità prodotta**, sommato ad un **costo variabile** *proporzionale alla quantità* prodotta.

In tali casi abbiamo quindi un certa **funzione obiettivo** del tipo:

$$C(x) = \begin{cases} 0 & x = 0 \\ ax + b & x > 0 \end{cases}$$

Per **modellare la discontinuità** e ottenere una *formulazione comprensibile dal risolutore* dobbiamo **assumere** che la **variabile sia limitata**:

$$0 < \epsilon \leq x \leq U \ , \epsilon, U \in \mathbb{R}$$

Intuitivamente, queste possono rappresentare delle **quantità minime e massime** di un bene che possono essere **prodotte**.

Ora, definite ϵ e U, possiamo procedere come segue:

Definizione (COSTI FISSI). Data la **funzione di costo** $C(x)$ ed i *bound* ϵ, U, definiamo una **variabile binaria** ausiliaria $y \in \{0,1\}$:

$$y = \begin{cases} 1 & x \geq \epsilon \\ 0 & \text{altrimenti} \end{cases}$$

E otteniamo la nuova formulazione del problema:

$$\begin{cases} C'(x) = ax + by \\ \epsilon y \leq x \\ x \leq Uy \\ y \in \{0, 1\} \end{cases}$$

Notiamo che:

Nota. Dai vincoli deduciamo:

- Se $y = 0$, i **vincoli schiacciano** x a **zero**.
- Se $y = 1$, allora x è **libera** di variare **entro i suoi bound**.

Ciò significa che la **variabile binaria modella** la **discontinuità** introdotta dal **costo fisso** e $x \in \{0\} \cup [\epsilon, U]$.

4.3.4 Disgiunzioni

Abbiamo visto che una **soluzione ammissibile** di un problema di ottimizzazione è una **soluzione che rispetta tutti i vincoli**; ciò significa che i vincoli sono *implicitamente legati* da un **operatore logico AND**.

A volte, tuttavia, vogliamo imporre che venga **rispettato almeno uno** tra un **gruppo di vincoli** alternativi: si parla in tal caso di **disgiunzioni**. Consideriamo per esempio:

$$a_1^\top x \leq b_1 \vee a_2^\top x \leq b_2 \, , l \leq x \leq u$$

Anche in questo caso, introduciamo delle **variabili binarie** y_1, y_2 tali che:

$$y_i = \begin{cases} 1 & a_i^\top x \leq b_i \\ 0 & \text{altrimenti} \end{cases} , y_1 + y_2 = 1$$

Definite queste, modelliamo il problema con la **tecnica Big-M**:

Definizione (DISGIUNZIONI). Dato un problema P, $l \leq x \leq u$, ed una **lista di vincoli alternativi**:

$$a_1^\top x \leq b_1 \vee a_2^\top x \leq b_2 \vee \ldots \vee a_k^\top x \leq b_k$$

Definiamo k **variabili binarie** y_i:

$$y_i = \begin{cases} 1 & a_i^\top x \leq b_i \\ 0 & \text{altrimenti} \end{cases} , \sum_{i=1}^{k} = 1$$

E **riscriviamo i vincoli** coinvolti come segue:

$$\begin{cases} a_i^\top x \leq b_i + M(1 - y_i) & i = 1, \ldots k \\ \sum_{i=1}^{k} y_i = 1 \\ l \leq x \leq u \end{cases}$$

Con $M \gg b_i \; \forall i = 1, \ldots, k$.

Notiamo che M è una **costante** positiva **sufficientemente grande** da **superare qualsiasi valore raggiungibile** dalle **espressioni dei vincoli**.

Nota (BIG-M). Di conseguenza, se il modello **sceglie** $y_s = 1$, il **termine corrispondente** $M(1 - y_s) = 0$ si **annulla** ed il **vincolo** associato **torna alla forma originale** e *deve quindi essere rispettato*.

Per **tutti gli altri indici**, invece, i termini $M(1-y_s) = M$ **rimangono** e **rendono *inutili* i vincoli** corrispondenti (che saranno *rispettati automaticamente* visto l'*alto valore di M*).

4.3.5 Funzione linare a tratti

Spesso, la **funzione obiettivo** potrebbe essere **definita solo per** k **punti notevoli** (chiamati *breakpoints* o *vertici*) con **coordinate** $a_1, a_2, \ldots, a_k$ (ricordiamo che in generale queste sono *vettori*, noi consideriamo il *caso unidimensionale* per la semplcità esplicativa) e **valori corrispondenti** $C(a_1), C(a_2), \ldots, C(a_k)$.

In tali casi, **dividiamo l'asse** in $k-1$ **intervalli** e modelliamo la situazione con una **funzione lineare a tratti** (o *PWL*, *piecewise linear*) utilizzando **due tipi di variabili**:

- $z_i \in \{0, 1\}$ $(i = 1, \ldots, k-1)$: variabili **binarie** per **determinare l'intervallo attivo**.
- $\lambda_i \geq 0$ $(i = 1, \ldots, k)$: variabili per esprimere la **combinazione convessa** dei **punti nell'intervallo attivo**.

Definizione (F. LINEARE A TRATTI). Il modello matematico risulta essere:

$$\begin{cases} x = \sum_{i=1}^{k} a_i \lambda_i \\ y = \sum_{i=1}^{k} C(a_i) \lambda_i \\ \sum_{i=1}^{k} \lambda_i = 1 \\ \sum_{i=1}^{k-1} z_i = 1 \\ \lambda_1 \leq z_1 \\ \lambda_k \leq z_{k-1} \\ \lambda_i \leq z_{i-1} + z_i & \forall i = 2, \ldots, k-1 \\ z_i \in \{0, 1\} & \forall i = 1, \ldots, k-1 \\ \lambda_i \geq 0 & \forall i = 1, \ldots, k \end{cases}$$

La figura 4.7 mostra un esempio di modello di una **funzione lineare a tratti**.
Cerchiamo ora di capire *come funziona il modello della PWL* aiutandoci con la figura:

Nota (INTUIZIONE PWL). Osservando il modello della PWL definito sopra, notiamo che:

- $\sum z_i = 1$ impone la **selezione di *un solo* intervallo** (e.g. consideriamo $z_2 = 1$).
- Poi, z_2 **"lascia accesi" solo** λ_2 e λ_3, mentre per **tutti gli altri** λ_i vale $\lambda_i \leq 0$ (che, *in combinazione* con $\lambda_i \geq 0$ implica $\lambda_i = 0$).

In altre parole, le variabili λ_i sono **forzate ad annullarsi**, **tranne** quelle agli **estremi** dell'**intervallo considerato**.

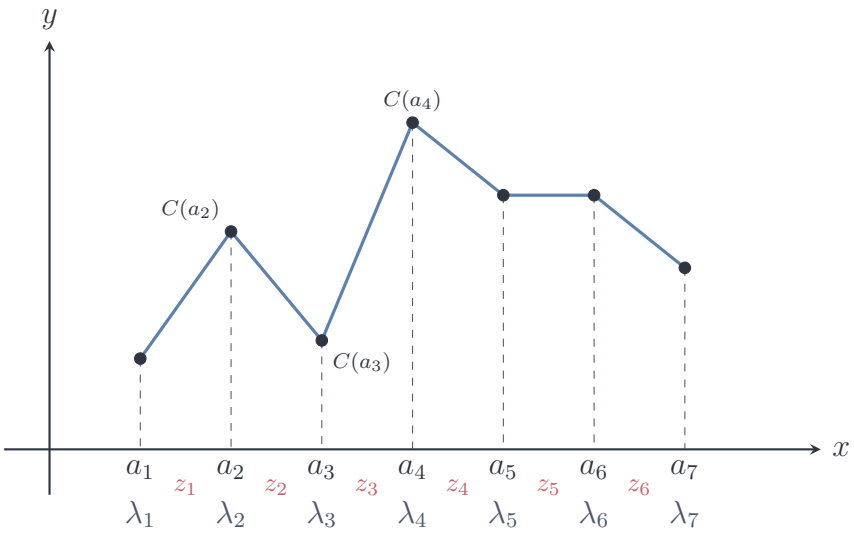


Figura 4.7: Esempio di funzione lineare a tratti

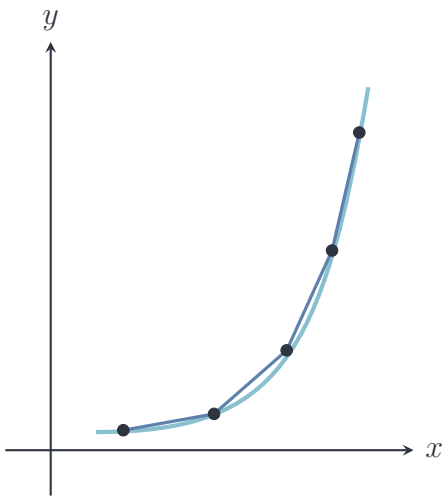


Figura 4.8: Approssimazione di una curva continua con una funzione PWL

Nota (APPROSSIMARE FUNZIONI CONTINUE). Le **funzioni lineari a tratti** possonoe essere **usate anche** per **approssimare delle funzioni continue**, come mostrato in figura 4.8.

4.4 Rappresentabilità MIP

Ci chiediamo a questo punto **quale classe di problemi** possiamo **modellare con il paradigma MIP**. Per rispondere a questa domanda, dobbiamo studiare la **caratterizzazione** dei **problemi MIP**.

Definizione (MIP-RAPPRESENTABILITA'). Sia $S \subseteq \mathbb{R}^n$ e sia $x \in S$. Allora l'insieme S si dice

MIP-rappresentabile se

$$S = \text{proj} \begin{cases} Ax + Bu + Dy \geq b \\ x_j \in \mathbb{Z} & j \in J \subseteq \{1, \dots, n\} \\ y \in \mathbb{Z} \end{cases}$$

Dove per "proj" intendiamo la **proiezione** dello *"spazio esteso"* (ottenuto con l'*aggiunta delle variabili* u e y) su $\mathbb{R}^n$ (spazio *"orginale"*). Ora, data questa definizione, possiamo vedere finalmente la **caratterizzazione dei problemi MIP**:

Teorema (TEOREMA DI JEROSLOW). Un insieme $S \subseteq \mathbb{R}^n$ è *MIP-rappresentabile* **se e solo se** è un'**unione finita** di **poliedri** con lo **stesso cono di reessione**.

Nota. Anche un **punto singolo** è considerabile **un poliedro**.

La figura 4.9 ci aiuta a capire che cosa si intende per **cono di recessione**.

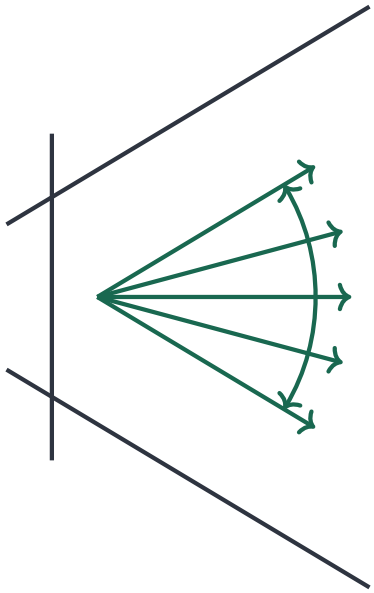


Figura 4.9: Cono di recessione di un poliedro

Definizione (CONO DI RECESSIONE DI UN POLIEDRO). E' l'insieme di **tutte le direzioni percorribili** all'**infinito senza mai uscire dal poliedro**.

Nota. Il **cono di recessione** di un **politopo** (*poliedro "chiuso"*) è semplicemente il **vettore nullo**.

Questa osservazione ci permette di concludere anche che:

Nota. L'**unione finita** di **politopi** è **MIP-rappresentabile**. Infatti, tutti hanno lo **stesso cono di recessione** (vettore nullo).

4.4.1 Problemi SAT

Ricordiamo la classe di problemi di **soddisfacibilità booleana** (problemi *SAT*):

> **Definizione** (PROBLEMI SAT). Problemi dati da n **variabili booleane** ed m **clausole booleane** (ovvero *disgiunzioni*). La **soluzione del problema** sta nel **determinare** se i **vincoli sono tutti soddisfacibili**.

Per capire meglio, vediamo un esempio:

> **Esempio.** Consideriamo per esempio il problema sulle variabili booleane:
>
> $$x_1, x_2, x_3, x_4$$
>
> Con le clausole seguenti:
>
> $$x_1 \vee \overline{x}_2 \vee x_3$$
> $$x_1 \vee x_4$$
> $$x_2 \vee x_3$$

Matematicamente, possiamo pensare di identificare una **variabile booleana** con sè stessa, la sua **negazione** con il suo *complementare a uno*, e possiamo verificare se le espressioni ottenute sono **vere** (≥ 1). Il problema dell'esempio sopra diventerebbe:

$$\begin{aligned} x_1 + (1 - x_2) + x_3 &\geq 1 \\ (1 - x_1) + x_4 &\geq 1 \\ x_2 + x_3 &\geq 1 \end{aligned}$$

> **Nota.** **Non** possiamo **risolvere un rilassamento continuo** di questi problemi. Basterebbe infatti **porre ogni variabile** ad $\frac{1}{2}$, ma tale soluzione **non ci darebbe nessuna informazione** utile.

Questo è il motivo per cui un problema del genere, **seppur facile da modellare** con il **paradigma MIP**, risulta **difficilmente risolvibile** in pratica.

4.4.2 Max-Cut

Vediamo ora un altro esempio di problema che *soffre dello stesso problema dei problemi SAT.*

> **Definizione** (MAX-CUT). Dato un **grafo** $G = (V, E)$, si dice **max-cut** una **partizione dei nodi** S (t.c. $S \cup V \setminus S = V$) di G tale che il **numero di archi** che **connettono nodi** in S a **nodi** in $V \setminus S$ sia **massimo**.

Possiamo **assegnare ad ogni nodo** i una **variabile booleana** x_i:

$$x_i = \begin{cases} 1 & i \in S \\ 0 & i \notin S \end{cases}$$

Potremmo modellare il problema come segue:

$$\begin{cases} \max \sum_{(i,j) \in E} [x_i(1 - x_j) + x_j(1 - x_i)] & \\ x_i \in \{0, 1\} & \forall i \in V \end{cases}$$

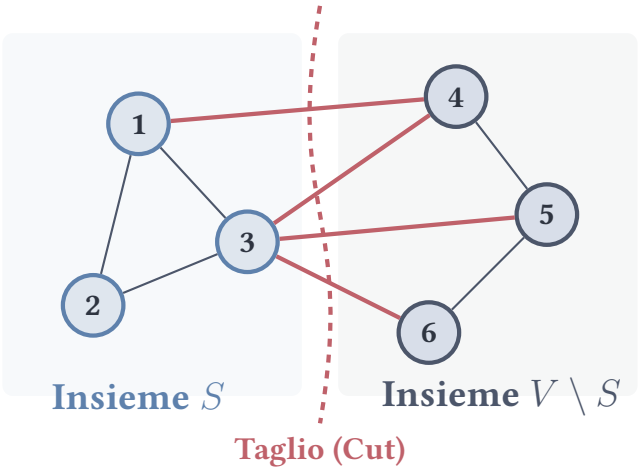


Figura 4.10: Problema Max-Cut

> **Nota.** La **funzione obiettivo *non* è lineare**.

Effettuiamo allora una **linearizzazione**, come segue:

$$\begin{cases} \max \sum_{(i,j)\in E} [x_i + x_j - 2y_{ij}] & \\ x_i \in \{0,1\} & \forall i \in V \\ y_{ij} \in \{0,1\} & \forall (i,j) \in E \\ y_{ij} = x_i \wedge x_j & \end{cases}$$

Dove l'ultimo vincolo viene "espanso" come segue:

$$\begin{cases} y_{ij} \leq x_i \\ y_{ij} \leq x_j \\ y_{ij} \geq x_i + x_j - 1 \end{cases}$$

> **Nota.** Questo è stato **possibile solo perchè entrambe le variabili sono binarie**, per cui la loro **moltiplicazione** è **equivalente ad un *AND* logico**, che abbiamo poi espanso in una serie di vincoli lineari.

> **Nota.** Come nel caso dei problemi SAT, anche in questo caso **non possiamo** fare **affidamento ad un rilassamento continuo** del problema: come prima, il risolutore produrrebbe una soluzione priva di senso in relazione al problema originale.

4.4.3 Casi patologici

Gli **esempi** di problemi che abbiamo visto sopra **sembrano fatti apposta** per essere modellati con il **paradigma MIP**, eppure risultano **difficili da risolvere in pratica**.

Dobbiamo comunque notare che si tratta di **casi patologici**.

> **Nota** (CASI PATOLOGICI E APPLICABILITA' PARADIGMA MIP)**.** Nella pratica, il **paradigma MIP** permette di **modellare**, e anche **risolvere efficientemente**, una **vasta gamma di problemi**.

Quelli che abbiamo visto sopra sono tranquillamente considerabili *casi "patologici"*, che *non rispecchiano* veramente l'applicabilità effettiva del paradigma.

5 PROBLEMI DI MODELLAZIONE

5.1 Knapsack

Il problema è il seguente:

Esempio (KNAPSACK). Immaginiamo di avere

- n **oggetti**, ciascuno con
  - Un **peso** w_j.
  - Un **profitto** p_j

  Con $j \in J = \{1, \dots, n\}$
- Un **contenitore** (*knapsack*) di **capacita massima** W.

Vogliamo capire come **massimizzare il profitto** rimanendo **entro i limiti di capacita**.

Introduciamo le **variabili binarie** seguenti:

$$x_j = \begin{cases} 1 & \text{se item j viene messo nel contenitore} \\ 0 & \text{altrimenti} \end{cases}$$

Il problema puo quindi essere formalizzato come segue:

$$\begin{cases} \max \sum_{j \in J} p_j x_j \\ \sum_{j \in J} w_j x_j \leq W \\ x_j \in \{0,1\} \qquad\qquad \forall j \in J \end{cases}$$

5.2 Facility location

Il problema è il seguente:

Esempio (FACILITY LOCATION). Abbiamo:

- Un **insieme** J di n **utenti**.

- Un **insieme** I di m **facilities**, ciascuna con un **costo fisso di costruzione** d_i.
- Ciascun **utente j comporta** un **costo di servizio** per poter **usufruire** della **i-esima facility** pari a c_{ij}.

Vogliamo **pianificare** la **costruzione delle facilities** in modo da **minizzare i costi di servizio**.

Introduciamo le variabili binarie seguenti:

$$y_i = \begin{cases} 1 & \text{se costruiamo la facility i} \\ 0 & \text{altrimenti} \end{cases}$$

Ed anche:

$$x_{ij} = \begin{cases} 1 & \text{se utente j e servito dalla facility i} \\ 0 & \text{altrimenti} \end{cases}$$

Il modello risulta pertanto essere:

$$\begin{cases} \min \sum_{i \in I} d_i y_i + \sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij} & \\ \sum_{i \in I} x_{ij} = 1 & \\ x_{ij} \leq y_i & \forall i \in I \, , \forall j \in J \\ y_i \in \{0,1\} & \forall i \in I \\ x_{ij} \in \{0,1\} & \forall i \in I \, , \forall j \in J \end{cases}$$

Nota (VINCOLI). Il vincolo

$$x_{ij} \leq y_i \; \forall i \in I \, , \forall j \in J$$

sta ad indicare il fatto che:

$$\begin{aligned} y_i = 0 &\implies x_{ij} = 0 \, , \forall j \in J \\ x_{ij} = 1 &\implies y_i = 1 \, , \forall i \in I \end{aligned}$$

In **totale**, questi vincoli sono $n \cdot m$ in **numero**.

Ci chiediamo allora se *possiamo fare di meglio*, ovvero se possiamo *rappresentare la stessa implicazione* usando *meno vincoli*. La risposta è **sni**: possiamo scrivere i vincoli come

$$\sum_{j \in J} x_{ij} \leq n y_i \, , \forall i \in I$$

che però comporta un **problema pratico**:

Nota (COMPROMESSO). Abbiamo **ridotto** il **numero di questi vincoli** da $n \cdot m$ a m.

Tuttavia, questa soluzione **risulta peggiore in pratica** in quanto **peggiora l'efficienza dei rilassamenti**. Inoltre, e facile intuire che i **vincoli aggregati** che abbiamo scritto sarebbero stati **deducibili anche dai vincoli singoli**.

5.3 Set covering

Il problema è il seguente:

Esempio (SET COVERING). Abbiamo come **input**:

- un **insieme** I detto **ground set**.
- Una **famiglia** $\mathcal{F} = \{F_1, \dots, F_n\}$ dove $F_j \subseteq I \; \forall j \in J = \{1, \dots, n\}$.

Vogliamo **scegliere un sottoinsieme** $\mathcal{F}^* = \{F_k\}$ con $k \in K \subseteq J$ di $\mathcal{F}$ tale per cui

$$\bigcup_{k \in K} F_k = I$$

Definiamo le **variabili binarie** seguenti:

$$x_j = \begin{cases} 1 & \text{se scelgo } F_j \\ 0 & \text{altrimenti} \end{cases}$$

Il modello risulta quindi essere:

$$\begin{cases} \min \sum_{j \in J} x_j \\ \sum_{j \,:\, i \in F_j} x_j \geq 1 & \forall i \in I \\ x_j \in \{0, 1\} & \forall j \in J \end{cases}$$

Nota (APPLICAZIONI REALI). Problemi di questo tipo sono **risolti giornalmente** dalle **compagnie aeree** per **determinare efficientemente** i **turni** dei **piloti**.

Nota (SET PARTIONING). A volte il problema puo essere modellato come problema di **set partionining**, in cui il **ground set** deve esere **coperto senza sovrapposizioni**. Chiaramente, questa formulazione risulta *piu difficile da risolvere*.